BLE-Based Asset Tracking with Edge Alerting

Student: Amina Nurlybekova

Project for the course: Industrial IoT

Date: June 2025

Table of Contents

1. 1. Introduction
2. 2. System Architecture
3. 3. Implementation
4. 4. Installation and Execution
5. 5. Example Output
6. 6. Conclusion
7. 7. References

1. Introduction

This project presents a BLE-based asset tracking system that uses RSSI threshold detection to identify movement and publishes alerts through MQTT. The system is intended for industrial and logistics applications where real-time tracking of object displacement is critical. In the modern world, Internet of Things (IoT) technologies are increasingly being implemented in logistics, manufacturing, office spaces and retail. One of the key areas is asset tracking, a task that requires reliable and energy-efficient technology.

Bluetooth Low Energy (BLE) is a lightweight wireless communication protocol specifically designed to operate with low power consumption. It is used for tracking, monitoring and identifying objects in closed and open spaces.

The "BLE-Based Asset Tracking with Edge Alerting" project implements a system that can track the movement of objects, analyze the signal strength (RSSI) and generate alerts if the object goes beyond the permissible zone. A special feature of the project is the ability to integrate

with physical BLE devices, as well as support for simulation for testing and debugging.

2. System Architecture

The system consists of several components:
- BLE tags (simulated or real)
- Edge server processing RSSI signals
- MQTT broker (e.g., Mosquitto)
- Subscriber receiving alerts

The edge server receives RSSI values and compares them with a threshold. If a tag moves away, an alert is sent.

3. Implementation

The system is implemented in Python and divided into modules:
- `ble_simulator.py`: generates RSSI values
- `edge_server.py`: checks for movement
- `mqtt_alert_publisher.py`: publishes alerts to MQTT
- `run_tracking.py`: the main simulation script

The `bleak` library is also used to connect to real BLE devices.

4. Installation and Execution

1. Install dependencies:
   ```bash
   pip install -r requirements.txt
   ```
2. Ensure Mosquitto is running:
   ```bash
   mosquitto
   ```
3. Run the simulation:
   ```bash
   ```

python3 src/run_tracking.py
```

## 5. Example Output

The system prints RSSI values. When the value falls below the threshold, an alert message appears:
`[ALERT] BLE_TAG_1 moved! RSSI: -89 dBm at 2025-06-05T14:32:20.420Z`

Example of output in simulation mode:

```
[ALERT] BLE_TAG_1 moved! RSSI: -82 dBm at 2025-06-05T18:02:34.881Z
BLE_TAG_2: RSSI = -64 dBm
BLE_TAG_3: RSSI = -66 dBm
```

Example of MQTT message:

```
{
  "alert": "Asset moved!",
  "tag_id": "BLE_TAG_1",
  "rssi": -82,
  "timestamp": "2025-06-05T18:02:34.881Z"
}
```

6. Conclusion

The project demonstrates basic principles of BLE interaction, MQTT communication, and edge logic implementation for real-time signal processing. The system is scalable and can be extended to support geolocation, alert storage, and integration with monitoring platforms.

7. References

- https://mqtt.org
- https://pypi.org/project/bleak/
- https://mosquitto.org
- Python documentation
- Industrial IoT course lectures
```